



Smart Contract Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
4 Code Overview	_____
4.1 Contracts Description	_____
4.2 Visibility Description	_____
4.3 Vulnerability Summary	_____
5 Audit Result	_____
6 Statement	_____

1 Executive Summary

On 2025.04.14, the SlowMist security team received the DMZ Finance team's security audit application for DMZ Token, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

This is the DMZ comprehensive identity-verified token system with compliance features. The IdentityRegistry contract manages user identities with country information and wallet associations, while MyToken implements an ERC20 token with enhanced compliance controls including country-based whitelisting/blacklisting. Users can register their identities with proper verification, link multiple wallets to a single identity, and then participate in the token economy where transfers are restricted based on regulatory status. The system includes advanced governance features like admin-controlled minting/burning requests, account freezing capabilities, and identity expiration dates, making it suitable for regulated financial environments where KYC (Know Your Customer) and geographic restrictions must be enforced.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Risk of excessive authority	Authority Control Vulnerability Audit	Medium	Acknowledged
N2	Risk of request ID collision	Design Logic Audit	Low	Fixed
N3	Risk of approving non-existent requests	Design Logic Audit	Low	Fixed
N4	Inconsistency between admin list and AccessControl roles	Design Logic Audit	Low	Fixed
N5	Redundant Code	Others	Suggestion	Fixed
N6	Missing Expiration Validation in Identity Management Functions	Others	Suggestion	Fixed
N7	Order Disruption in Array Removal Functions	Others	Suggestion	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

<https://github.com/dmzFinance/dmz>

commit: 3a76b93b47918b53c8977fb76b0d8f3f98717dbe

Audit Scope:

```

├─ IIdentityRegistry.sol
├─ IdentityRegistry.sol

```

MyToken.sol

Fixed Version:

<https://github.com/dmzFinance/dmz>

commit: 9ef658d77c6439e623840cbd38207eac9aa8e9d5

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

MyToken			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	ERC20 Ownable ERC20Permit
decimals	Public	-	-
pause	Public	Can Modify State	onlyOwner
unpause	Public	Can Modify State	onlyOwner
mintRequest	Public	Can Modify State	-
burnRequest	Public	Can Modify State	-
approveRequest	Public	Can Modify State	onlyRole
rejectRequest	Public	Can Modify State	onlyRole
getRequest	Public	-	-
forcedTransfer	Public	Can Modify State	onlyOwner
freezeAccount	Public	Can Modify State	onlyOwner
unfreezeAccount	Public	Can Modify State	onlyOwner
_verifyWhitelisted	Internal	-	-

MyToken			
_isCountryInList	Private	-	-
_update	Internal	Can Modify State	-
addFundManager	Public	Can Modify State	-
removeFundManager	Public	Can Modify State	-

IdentityRegistry			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
registerIdentity	External	Can Modify State	onlyAdmin
registerIdentities	External	Can Modify State	onlyAdmin
deleteIdentity	External	Can Modify State	onlyAdmin
addAdmin	External	Can Modify State	onlyAdmin
isAdmin	Public	-	-
removeAdmin	External	Can Modify State	onlyAdmin
addWallets	External	Can Modify State	onlyAdmin
removeWallets	External	Can Modify State	onlyAdmin
updateExpiryDate	External	Can Modify State	onlyAdmin
updateCountry	External	Can Modify State	onlyAdmin
updateData	External	Can Modify State	onlyAdmin
verifyAddress	External	-	-
getIdentityDetails	External	-	-

4.3 Vulnerability Summary

[N1] [Medium] Risk of excessive authority**Category: Authority Control Vulnerability Audit****Content**

1. In the MyToken contract, the owner can force the transfer of any user's token to any address through the forcedTransfer function without the user's consent. And the owner role can approve or reject any mint or burn request through the mintRequest and burnRequest functions.

Code location:

MyToken.sol#LL154-L156, L121-L147

```
function forcedTransfer(address from, address to, uint256 amount) public onlyOwner
{
    _transfer(from, to, amount);
}
```

2. The admin role can register any identities and also can invalidate the identities.

Solution

In the short term, transferring owner ownership to multisig contracts is an effective solution to avoid single-point risk. But in the long run, it is a more reasonable solution to implement a privilege separation strategy and set up multiple privileged roles to manage each privileged function separately. And the authority involving user funds should be managed by the community, and the EOA address can manage the authority involving emergency contract suspension. This ensures both a quick response to threats and the safety of user funds.

Status

Acknowledged; After communicating with the project team, they stated that the forcedTransfer design is a compliance requirement.

[N2] [Low] Risk of request ID collision**Category: Design Logic Audit****Content**

In the MyToken contract, the generation of request IDs for mint and burn operations is done using the following mechanism, which could potentially lead to collisions: `bytes32 requestID =`

`keccak256(abi.encodePacked(block.timestamp, msg.sender, _lastRequestID));` This method relies on three components: the block timestamp, the sender's address, and the most recently generated request ID. However, if a user submits multiple transactions (e.g., one mint request and one burn request) that get included in the same block, they will have the same `block.timestamp` and `msg.sender` values. If these transactions somehow end up using the same `_lastRequestID` value (for example, if they're processed concurrently by different nodes), they could generate identical request IDs. And if request ID collisions occur, it could lead to incorrect processing of mint/burn operations.

Code location:

MyToken.sol#L79-L118

```
function mintRequest(address to, uint256 amount) public returns (bytes32) {
    ...
    bytes32 requestId = keccak256(abi.encodePacked(block.timestamp, msg.sender,
_lastRequestID));
    ...
    _lastRequestID = requestId;
    ...
}

function burnRequest(uint256 amount) public returns (bytes32) {
    ...
    bytes32 requestId = keccak256(abi.encodePacked(block.timestamp, msg.sender,
_lastRequestID));
    ...
    _lastRequestID = requestId;
    ...
}
```

Solution

It's recommended to implement a more robust ID generation mechanism, such as adding a monotonically increasing counter and including the request type as part of the input to the hash function, to ensure that different operations from the same user will generate different IDs.

Status

Fixed

[N3] [Low] Risk of approving non-existent requests

Category: Design Logic Audit

Content

In the MyToken contract, the `approveRequest` and `rejectRequest` functions only check if the status of a request is Pending, but do not verify if the request actually exists. Since the default value for the status field in the `TokenRequest` struct is 0, which corresponds to `RequestStatus.Pending`, these functions could process non-existent requests.

Code location:

MyToken.sol#L121-147

```
function approveRequest(bytes32 requestID) public onlyRole(FUND_MANAGER_ROLE) {
    require(_requests[requestID].status == RequestStatus.Pending, "Request is not
pending");
    TokenRequest storage request = _requests[requestID];

    if (request.requestType == RequestType.Mint) {
        _mint(request.account, request.amount);
    } else if (request.requestType == RequestType.Burn) {
        _burn(address(this), request.amount);
        _temporaryBalances[request.account] -= request.amount;
    }
    request.status = RequestStatus.Approved;
    request.finalizedAt = block.timestamp;
    emit RequestApproved(requestID, block.timestamp, request.requestType);
}

function rejectRequest(bytes32 requestID) public onlyRole(FUND_MANAGER_ROLE) {
    require(_requests[requestID].status == RequestStatus.Pending, "Request is not
pending");
    TokenRequest storage request = _requests[requestID];

    if(request.requestType == RequestType.Burn) {
        _transfer(address(this), request.account, request.amount);
        _temporaryBalances[request.account] -= request.amount;
    }
    request.status = RequestStatus.Rejected;
    request.finalizedAt = block.timestamp;
    emit RequestRejected(requestID, block.timestamp, request.requestType);
}
```

Solution

It's recommended to add an additional check in the `approveRequest` and `rejectRequest` functions to verify that the

request actually exists.

Status

Fixed

[N4] [Low] Inconsistency between admin list and AccessControl roles

Category: Design Logic Audit

Content

In the IdentityRegistry contract, there are two separate mechanisms for tracking administrators:

The OpenZeppelin AccessControl's role-based system, which uses DEFAULT_ADMIN_ROLE

And a custom admins array that stores admin addresses. These two mechanisms can become out of sync if the standard AccessControl functions are called directly instead of through the contract's custom admin management functions.

Code location:

IdentityRegistry.sol#L109-L135

```
function addAdmin(address newAdmin) external onlyAdmin {
    require(!isAdmin(newAdmin), "Address is already an admin");
    grantRole(DEFAULT_ADMIN_ROLE, newAdmin);
    admins.push(newAdmin);
}

function isAdmin(address admin) public view returns (bool) {
    for (uint i = 0; i < admins.length; i++) {
        if (admins[i] == admin) {
            return true;
        }
    }
    return false;
}

function removeAdmin(address adminToRemove) external onlyAdmin {
    revokeRole(DEFAULT_ADMIN_ROLE, adminToRemove);
    for (uint256 i = 0; i < admins.length; i++) {
        if (admins[i] == adminToRemove) {
            admins[i] = admins[admins.length - 1];
            admins.pop();
            break;
        }
    }
}
```

```

    }
    emit AdminRemoved(adminToRemove);
}

```

Solution

It's recommended to remove the custom admins array and rely solely on AccessControl or rewrite the grantRole and revokeRole functions to ensure both admin tracking mechanisms remain synchronized at all times.

Status

Fixed

[N5] [Suggestion] Redundant Code

Category: Others

Content

1. In the IdentityRegistry contract, both registerIdentity and registerIdentities functions contain redundant loops that iterate through the same arrays of wallet addresses, causing computational redundancy and unnecessary gas consumption. This represents a code efficiency issue. In both functions, the first loop checks if each wallet address is already registered. A second loop then sets the mapping relationship for the same wallet addresses. This implementation leads to duplicate array traversals, significantly increasing gas consumption.

Code Location:

IdentityRegistry.sol#L31-L94

```

function registerIdentity(
    ...
) external onlyAdmin {
    ...
    for (uint256 i = 0; i < wallets.length; i++) {
        require(addressToHashTx[wallets[i]] == bytes32(0), "One or more wallets
are already registered");
    }
    ...
    for (uint256 i = 0; i < wallets.length; i++) {
        addressToHashTx[wallets[i]] = hashTx;
    }
    ...
}

function registerIdentities(

```

```

    ...
} external onlyAdmin {
    ...
    for (uint256 j = 0; j < hashTx.length; j++) {
        ...
        for (uint256 i = 0; i < walletsLists[j].length; i++) {
            require(addressToHashTx[walletsLists[j][i]] == bytes32(0), "One or
more wallets are already registered");
        }
        ...
        for (uint256 i = 0; i < walletsLists[j].length; i++) {
            addressToHashTx[walletsLists[j][i]] = hashTx[j];
        }
        ...
    }
}

```

2. In the IdentityRegistry contract, the verifyAddress function contains redundant verification logic that increases gas consumption without providing additional security or functionality:

The function first checks if the wallet is registered using the addressToHashTx mapping

If the wallet is registered, it retrieves the associated identity and checks if it's expired

Then, it unnecessarily loops through the identity's wallet array to check if the wallet exists, even though this was already confirmed by the mapping lookup.

Code Location:

IdentityRegistry.sol#L191-L208

```

function verifyAddress(address walletToVerify) external view returns (bool,
bytes32) {
    bytes32 hashTx = addressToHashTx[walletToVerify];
    ...
    for (uint256 i = 0; i < identity.wallets.length; i++) {
        if (identity.wallets[i] == walletToVerify) {
            return (true, identity.country);
        }
    }
    return (false, bytes32(0));
}

```

Solution

1. It's recommended to merge the redundant loops in both functions to optimize gas usage. This also prevents duplicate wallets[] addresses from existing in the registered wallet addresses.
2. It's recommended to remove the redundant loop and simplify the function.

Status

Fixed

[N6] [Suggestion] Missing Expiration Validation in Identity Management Functions

Category: Others

Content

Multiple functions in the IdentityRegistry contract only check if an identity exists (`expiryDate > 0`) but fail to validate if the identity has already expired. This allows administrators to perform operations on expired identities, which could lead to inconsistent states and wasted resources:

1. Five functions (`addWallets`, `removeWallets`, `updateExpiryDate`, `updateCountry`, `updateData`) only check if the identity exists but not if it's expired.
2. The `updateExpiryDate` function specifically fails to validate that the new expiry date is in the future.
3. It doesn't check the `maxWalletsPerIdentity` is greater than 0.

Code location:

IdentityRegistry.sol#L28, L136-L189

```

constructor(uint256 _maxWalletsPerIdentity) {
    ...
    maxWalletsPerIdentity = _maxWalletsPerIdentity;
}

function addWallets(bytes32 hashTx, address[] memory newWallets) external
onlyAdmin {
    Identity storage identity = identities[hashTx];
    require(identity.expiryDate > 0, "Identity not found");
    ...
}

function removeWallets(bytes32 hashTx, address[] memory walletsToRemove) external
onlyAdmin {
    ...
    require(identity.expiryDate > 0, "Identity not found");
}

```

```

    ...
}

function updateExpiryDate(bytes32 hashTx, uint256 newExpiryDate) external
onlyAdmin {
    Identity storage identity = identities[hashTx];
    require(identity.expiryDate > 0, "Identity not found");
    identity.expiryDate = newExpiryDate;
    emit ExpiryDateUpdated(hashTx, newExpiryDate);
}

function updateCountry(bytes32 hashTx, bytes32 country) external onlyAdmin {
    Identity storage identity = identities[hashTx];
    require(identity.expiryDate > 0, "Identity not found");
    ...
}

function updateData(bytes32 hashTx, bytes32 data) external onlyAdmin {
    Identity storage identity = identities[hashTx];
    require(identity.expiryDate > 0, "Identity not found");
    ...
}

```

Solution

1,2 It's recommended to add proper validation to ensure operations are only performed on valid (non-expired) identities and that new expiry dates are set in the future.

3. It's recommended to add proper validation to ensure maxWalletsPerIdentity is greater than 0.

Status

Fixed

[N7] [Suggestion] Order Disruption in Array Removal Functions

Category: Others

Content

In the IdentityRegistry contract, both removeWallets and removeAdmin functions use a "swap and pop" pattern to remove elements from arrays. This pattern changes the relative order of elements in the arrays, which could lead to unexpected behavior:

The functions replace the element to be removed with the last element in the array, then pop the last element. This implementation changes the relative order of elements in the array. If there is any logic or integrations that depend on

the order of wallets or admins may be affected.

Code Location:

IdentityRegistry.sol#L125-L135, L151-L168

```
function removeAdmin(address adminToRemove) external onlyAdmin {
    revokeRole(DEFAULT_ADMIN_ROLE, adminToRemove);
    for (uint256 i = 0; i < admins.length; i++) {
        if (admins[i] == adminToRemove) {
            admins[i] = admins[admins.length - 1];
            admins.pop();
            break;
        }
    }
    emit AdminRemoved(adminToRemove);
}

function removeWallets(bytes32 hashTx, address[] memory walletsToRemove) external
onlyAdmin {
    ...
    for (uint256 j = 0; j < walletsToRemove.length; j++) {
        ...
        while (index < identity.wallets.length && identity.wallets[index] !=
walletsToRemove[j]) {
            index++;
        }
        require(index < identity.wallets.length, "One or more wallets not found in
this identity");
        identity.wallets[index] = identity.wallets[identity.wallets.length - 1];
        identity.wallets.pop();
        delete addressToHashTx[walletsToRemove[j]];
    }
    emit WalletsRemoved(hashTx, walletsToRemove);
}
```

Solution

It's recommended to use an order-preserving removal approach if array order is important.

Status

Acknowledged

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002504160001	SlowMist Security Team	2025.04.14 - 2025.04.16	Medium Risk

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project. During the audit work, we found 1 medium risk, 3 low risks, and 3 suggestions. All the findings were fixed or acknowledged. The code was not deployed to the mainnet. Since the protocol has not yet been deployed on the mainnet and the issue of excessive privileges within the protocol remains unresolved, the final conclusion is still assessed as medium risk.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>